

Virtual Memory

元培数科 王恩博

Let's get started →

2025/12/3



Knowledge Review

虚拟内存

Virtual Memory

为什么要使用虚拟内存？

虚拟内存是一层很关键的抽象：

- 它隐藏了物理内存的细节，为程序提供了一个抽象的、一致的内存视图
- 它保证了每个进程地址空间的独立性，使得进程之间不会相互干扰
- 可以分配超过物理内存的地址空间，充分利用物理内存（物理地址是唯一的，不能多次分配）

物理寻址和虚拟寻址

物理寻址

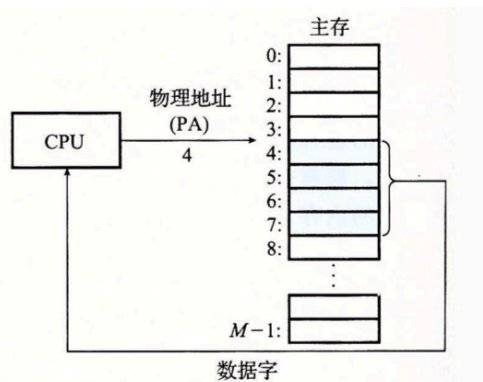


图 9-1 一个使用物理寻址的系统

1. 获得物理地址 (PA, Physical Address)
2. 通过地址总线访问物理内存

物理地址不要求总量 M 是 2 的幂次，寻址范围 $0 \sim M - 1$

虚拟寻址

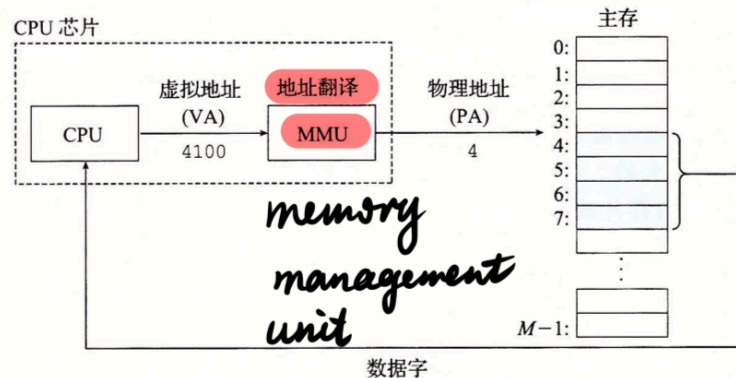


图 9-2 一个使用虚拟寻址的系统

1. 获得虚拟地址 (VA, Virtual Address)
2. 通过 CPU 中的内存管理单元 (MMU) 的转换，将虚拟地址转换为物理地址
3. 通过地址总线访问物理内存

虚拟地址要求总量 N 是 2 的幂次，即满足 $N = 2^n$ ，寻址范围 $0 \sim N - 1 / 0 \sim 2^n - 1$

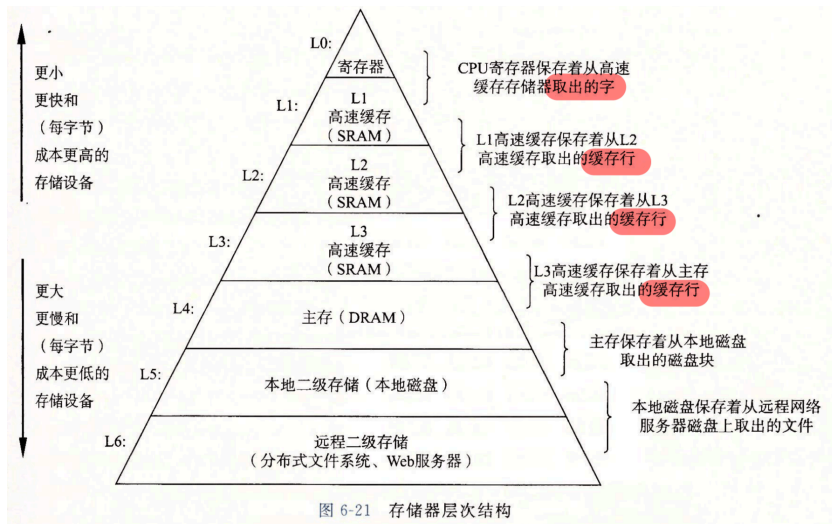
虚拟内存做为缓存的工具

Virtual Memory as a tool for caching

回想我们在第六章学习过的内容：

访问更低级存储设备前，我们会先访问更高一级存储设备，试图命中缓存。

由于使用虚拟内存寻址时，我们会先访问虚拟内存，再访问物理内存，所以我们可以将先前是否缓存的检查放到虚拟内存这一层。



虚拟内存做为缓存的工具

Virtual Memory as a tool for caching

任何时刻，虚拟内存的集合分为三个不相交的子集：

- **未分配的**：完全没在虚拟内存中分配（用不到、省下来）
- 已分配的
 - **已缓存的**：已经在物理内存中缓存，不需要到到硬盘中取
 - **未缓存的**：不在物理内存中缓存，需要到硬盘中取

和之前在缓存中最小交换的单位是 **块** 一样，虚拟内存中最小交换的单位是 **页**。

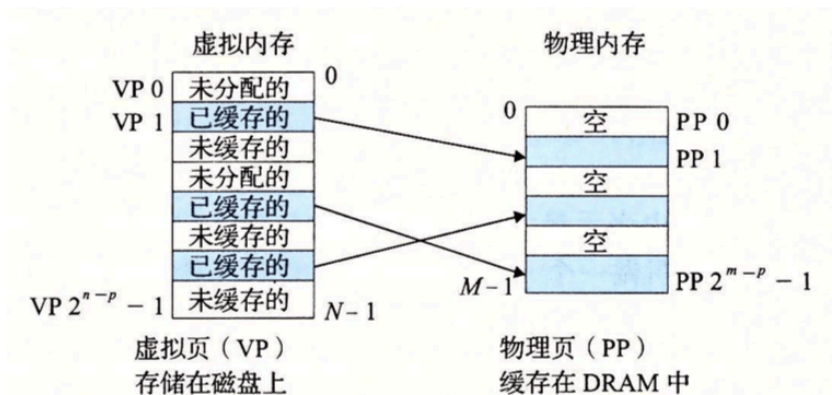


图 9-3 一个 VM 系统是如何使用主存作为缓存的

页表

Page Table

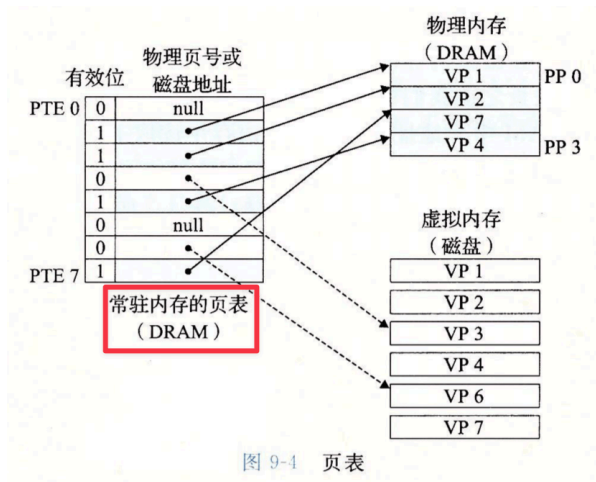
由于现在虚拟内存参与到了缓存机制中，所以和之前的缓存一样，虚拟内存也需要一个方式来判断一个页是否完成了缓存，而这，就是页表。

页表是一个页表条目（Page Table Entry, PTE）的数组。

每个页表条目由一个有效位（valid bit）和一个 n 位地址字段组成：

有效位：判断一个页是否在物理内存中

- 0：不在物理内存中，对应的页表项为：
 - 空（未分配，NULL）
 - 虚拟页 起始地址（未缓存，对应更低级缓存（磁盘）中的起始地址）
- 1：已缓存，缓存在了物理内存中，对应项为 物理页 的起始地址



Key Points:

- 页表是常驻在物理内存中的（总是需要用页表来索引）。
- DRAM 缓存是全相联的，所以任意物理页都可以包含任意虚拟页。

页表索引

Page Table Indexing

成功

如果已缓存（有效位为 1），称为 **页命中**（Page Hit）：

直接从页表中对应的物理内存地址中读取数据即可

失败

如果未缓存（有效位为 0），称为 **缺页**（Page Fault）：

此时会触发一个 **缺页异常**（Page Fault）

回顾：缺页异常是？类型（同步 / 异步）、恢复到的地点是？
故障、同步、当前指令 I_{cur}

缺页异常调用内核中的缺页异常处理程序，选择一个 **牺牲页**（如果牺牲页被修改了，就写回磁盘）

然后用所需虚拟页替换之，并修改牺牲页和新虚拟页对应的页表条目。

分配页面：如果要分配一个新的虚拟内存页（如调用 `malloc`），则在磁盘上创建空间并更新对应的页表条目（PTE），使其指向新创建的页面。

局部性

Locality

从内存到磁盘的交换时间很长会导致高昂的不命中处罚。

但是，由于页表很大，所以会具有比较好的空间局部性。

抖动（thrashing）：如果工作集的的大小超出了物理内存的大小，那么这时页面还是会不断地换进换出。

虚拟内存用作内存管理

Virtual Memory as a Memory Management Tool

全局单页表 → 每个进程拥有一个独立的页表。

提供了一个 **假象**：每个进程独占所有物理内存。

- **简化链接**：每个进程使用了类似的内存格式，使得链接器的设计和实现被大大简化（不用考虑物理地址 PA，只用考虑虚拟地址 VA）
- **简化加载**：将 `.text` 和 `.data` 节加载到新创建的进程中时，只需要分配虚拟页并将页表条目指向目标文件中适当的位置
- **简化共享**：不同进程中适当的虚拟页面可以映射到相同的物理页面
- **简化内存分配**：物理页面可以随机地分散在物理内存中（这项单页表就有）

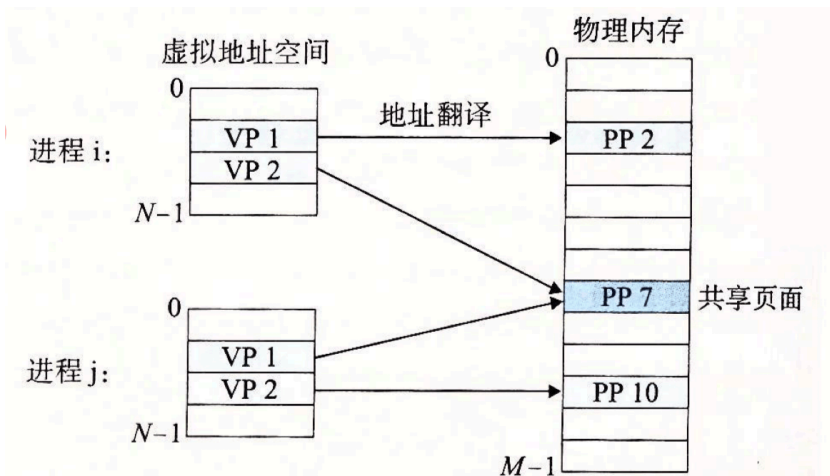


图 9-9 VM 如何为进程提供独立的地址空间。操作系统为系统中的每个进程都维护一个独立的页表

虚拟内存作为内存保护的工具有

Virtual Memory as a Memory Protection Tool

除了标明是否缓存的有效位，页表条目中还可以包含数个 **权限位 (protection bit)**，用于标明一个页的权限。

如果指令违反许可条件，CPU 会触发一个一般保护 **故障**，将控制传递给内核中的异常处理程序。

Linux Shell 一般将其报告为 **段错误 (Segmentation Fault)**。

虚拟地址翻译

Virtual Address Translation

回顾：缓存寻址

缓存	虚拟内存
有效位	页表有效位
标记位	/, 用于页表是全量的
块偏移	物理页偏移, 利用了交换块大小一致

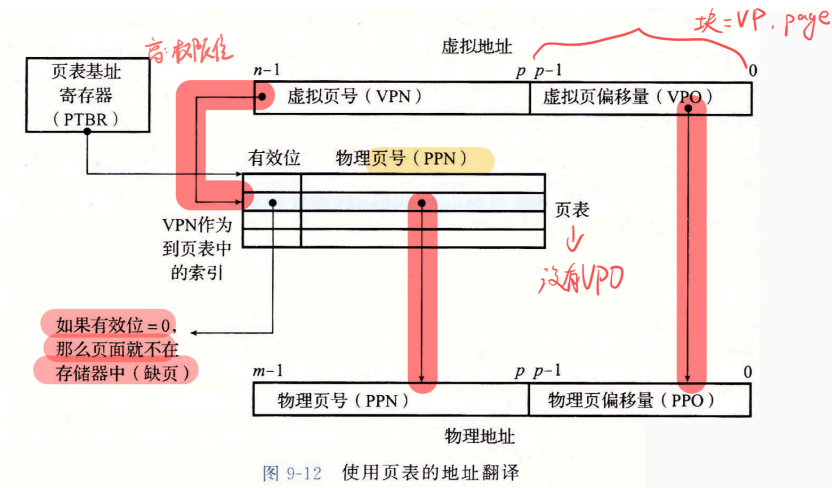


图 9-12 使用页表的地址翻译

- $VA = VPN$ (Virtual Page Number, 虚拟页号) : VPO (Virtual Page Offset, 虚拟页偏移), : 代表拼接
- VPN 直接作为索引去查找, 替代了标记位的对比判断, 经页表查询后得到物理页号 PPN
(不是之前说的物理地址偏移了, 但是是等价的, 因为 $PPN \times \text{页大小} = \text{物理地址偏移}$)
- VPO 等价于块偏移, 用于索引页内地址
- $PA = PPN : PPO$

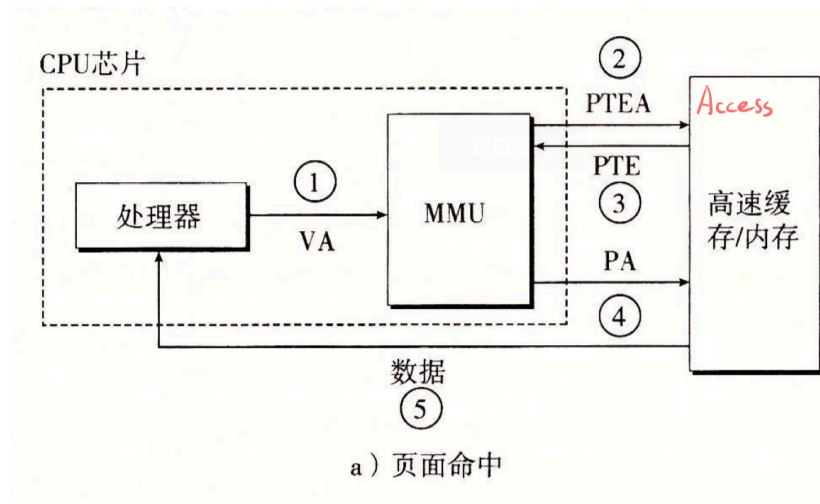
虚拟地址翻译

Virtual Address Translation

命中

1. 处理器生成一个虚拟地址，并把它传送给 MMU
2. MMU 生成 PTE 地址，并从高速缓存 / 主存请求得到它
3. 高速缓存 / 主存向 MMU 返回 PTE
4. MMU 构造物理地址，并把它传送给高速缓存 / 主存
5. 高速缓存 / 主存返回所请求的数据字给处理器

回忆：页表是常驻在物理内存中的，所以请求页表一定会触发一次访存，且（此时）必然命中。



虚拟地址翻译

Virtual Address Translation

缺页

1. 处理器生成一个虚拟地址，并把它传送给 MMU
2. MMU 生成 PTE 地址，并从高速缓存 / 主存请求得到它
3. 高速缓存 / 主存向 MMU 返回 PTE
4. MMU 触发异常，传递 CPU 中的控制到操作系统内核中的缺页异常处理程序
5. 缺页异常处理程序确定并处理牺牲块
6. 缺页处理程序调入新的页面，并更新内存中的 PTE

回忆：页表是常驻在物理内存中的，所以请求页表一定会触发一次访存，且（此时）必然命中。但是，页表不代表页在。

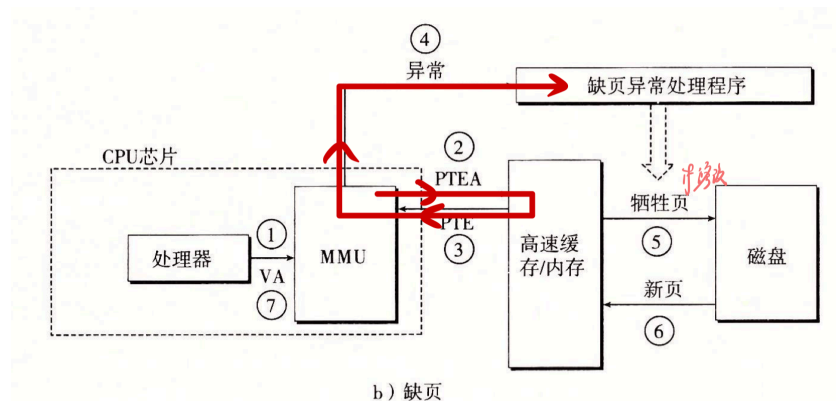


图 9-13 页面命中和缺页的操作图 (VA: 虚拟地址。PTEA: 页表条目地址。PTE: 页表条目。PA: 物理地址)

万物皆可缓存

Everything is Cacheable

- 页表常驻在物理内存中，还很大（总量： $N_{pages} \times bit_{PTE}$ ）
- CPU 到 内存 还是存在较大的延迟（回忆：在此之前我们会试图使用 L1 ~ L3 缓存来缓解）
- 页表本身也是数据，页表条目可以缓存，就像其他的数据字一样，也可以通过 L1 ~ L3 缓存来提高访问速度

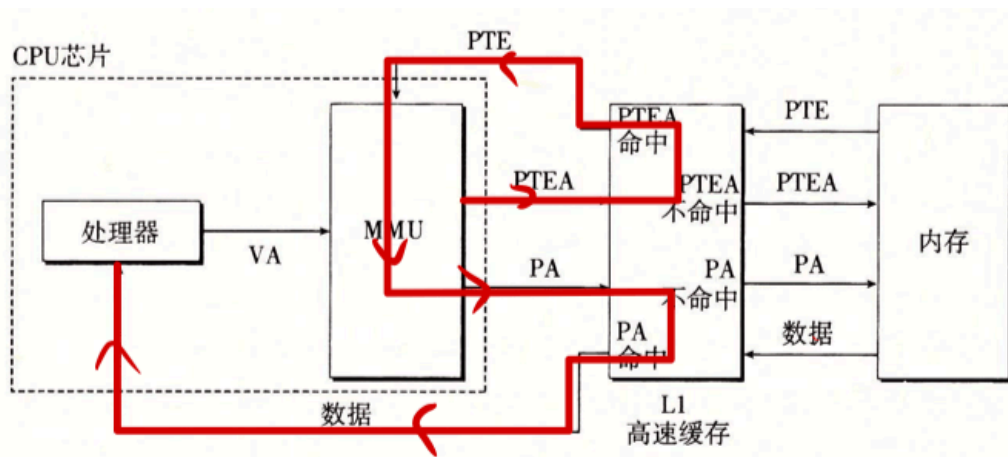


图 9-14 将 VM 与物理寻址的高速缓存结合起来 (VA: 虚拟地址。
PTEA: 页表条目地址。PTE: 页表条目。PA: 物理地址)

传统缓存是有极限的！

wobudangrenle!

- 每条指令都要访存，每次访存都要获取一个 PTE
- 即使 PTE 就在 L1 缓存中，每次访问也还是需要 1~2 个时钟周期，积沙成塔，性能损失很大，但这已经是把 PTE 当做传统数据字的极限了
- 继续优化，**添加新硬件**，再叠一层缓存来加速 PTE 的获取！

翻译后备缓冲器 TLB

Translation Lookaside Buffer, TLB

L1 ~ L3 缓存

- 把页表当做普通数据字来看待
- 它们的索引，操作的是这个数据字所在的内存物理地址
- 利用的是内存物理地址的局部性
- Cache 缓存物理地址，对应以物理地址开头的物理块，目的是加快数据字的访问

TLB 缓存

- 缓存的是物理地址
- 它的索引，操作的是虚拟地址 VP 中的 VPN
- 利用的是虚拟地址的局部性
- TLB 缓存虚拟地址，对应 PTE，目的是加速 VP 到 PP 的翻译

TLB 通常是 **高度相联的**，这带来了更高的灵活性。

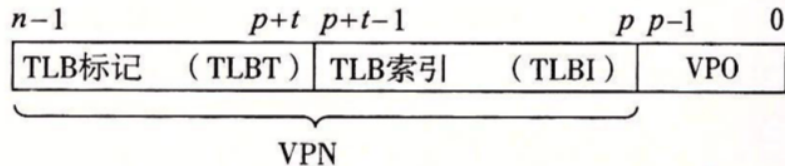
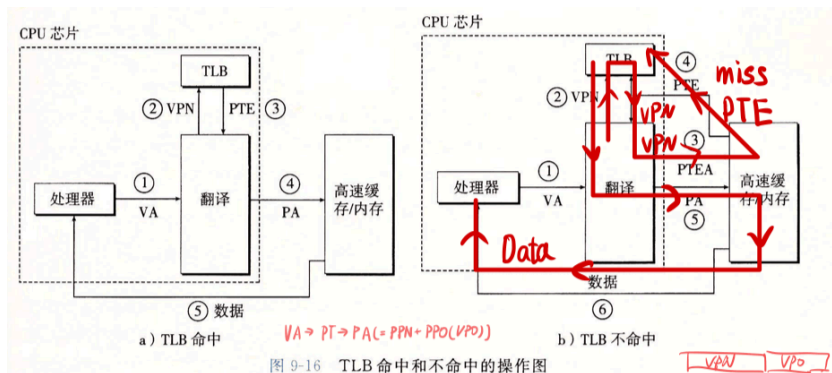


图 9-15 虚拟地址中用以访问 TLB 的组成部分

多级页表

Multi-level Page Table

页表常驻在物理内存中，还很大（总量： $N_{pages} \times bit_{PTE}$ ）

TLB 给我们带来了一些启发：我们可以利用 VP 的空间局部性（VPN 的时间局部性）。
连续的 VP，变动的是低阶位 VPO，VPN 基本不变。

那么，能不能进一步的利用 VPN 的空间局部性呢？

多级页表

Multi-level Page Table

多级页表：将 VPN 的前缀再拆分成数段 $VPN_i, i \in [1, k]$ ，假设每段有 n 位：

- 一级页表：VPN₁，对它而言，VPN_{2~k} : VPO 是它的“VPO”，它的每一项，存储的是一个 **2 级页表的基址**，可以认为它的每一项缓存了 $\frac{1}{2^n}$ 的虚拟内存 **(加起来寻址了全部)**
- 二级页表：VPN₂，对它而言，VPN_{3~k} : VPO 是它的“VPO”，它的每一项，存储的是一个 **3 级页表的基址**，可以认为它的每一项缓存了 $\frac{1}{2^{2n}}$ 的虚拟内存
- ...
- 第 k 级页表：VPN_k，对它而言，VPO 就是它的“VPO”，**也只有它的每一项，存储了实际的 PPN**，可以认为它的每一项，缓存了 $\frac{1}{2^{kn}}$ 的虚拟内存，这恰好等于 2^m ，其中 m 是 PPO 的位数

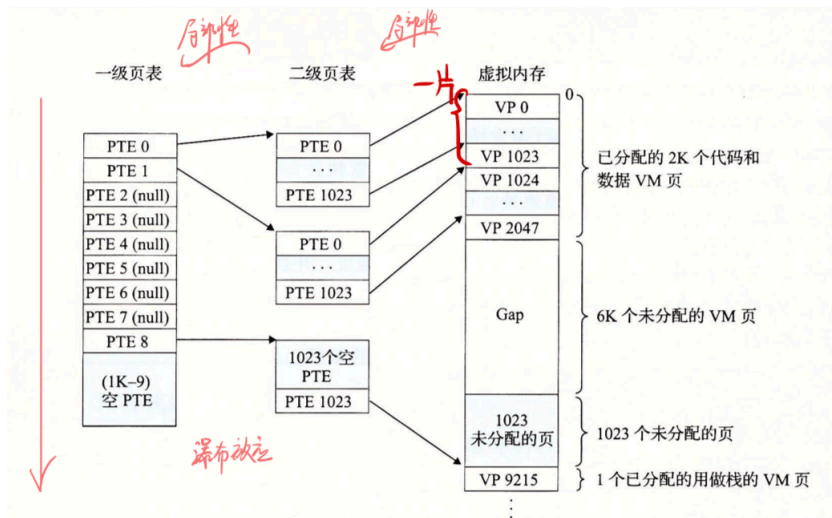


图 9-17 一个两级页表层次结构。注意地址是从上往下增加的

页表也是更

此时：**只有 1 级页表常驻在物理内存中**，2 级页表和 3 级页表，可以不在物理内存中。

一些延伸概念

Some Boring Things

1. PTE (Page Table Entry) :

- PTE 是页表中的一个条目，每个条目包含一个虚拟页到物理页的映射
- PTE 包含物理页框地址以及一些控制位（如有效位、读/写权限位、访问位等）
- 当 CPU 需要访问某个虚拟地址时，会通过页表找到对应的 PTE，从而得到物理地址

2. PDE (Page Directory Entry) : 类似 PTE，但是层级在 PTE 之上

- PDE 是页目录中的一个条目，每个条目指向一个页表
- 页目录用于组织和管理多个页表
- PDE 也包含一些控制信息，如有效位、访问权限等

一些延伸概念

Some Boring Things

3. 大页模式 (Huge Pages or Large Pages) :

往年题最喜欢的一集

- 大页模式是一种内存分页机制，其中每个页的大小比普通页更大
- 考虑右图，现在的寻址方式是，先用 VPN1 找到一级页表中的 PDE，再解码得到二级页表的基址，再用 VPN2 找到二级页表中的 PDE...
- 如果我们能保证连续的 VPN2 指向的三级页表基址是连续的，那么我们就可以减少页表的层级，用一个一级页表的 PDE 来表示一整个大页，这个大页融合了二级页表和三级页表
- 联想比较：`int a[b][c]` 和 `int a[b*t + c]`，利用二进制的特性
- 大页模式可以提高 TLB 的命中率，减少页表的查找次数

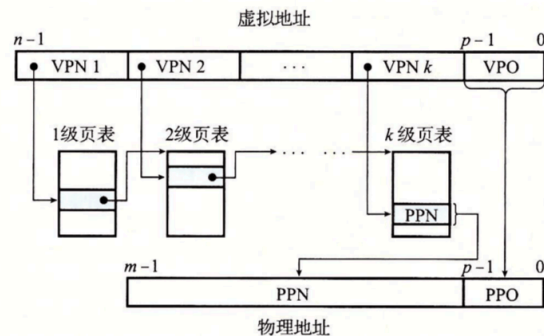


图 9-18 使用 k 级页表的地址翻译

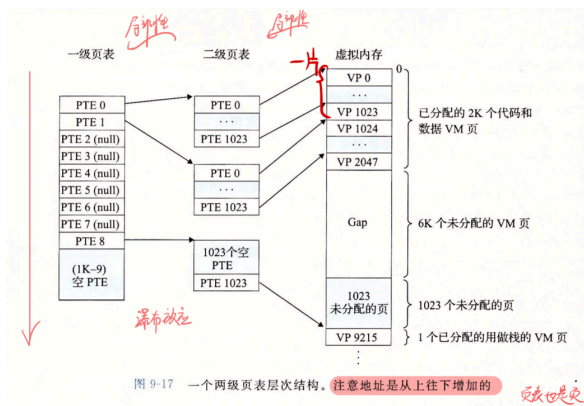


图 9-17 一个两级页表层次结构。注意地址是从上往下增加的

要点总结

Recap

基础概念：

- 虚拟内存三个便利：作为缓存工具、作为内存管理（简化链接，简化加载，简化共享，简化内存分配）、作为内存保护
- MMU 完成 地址翻译 + 权限保护
- 页表数据结构：用来管理 page 的状态、信息（利用对齐特性，低位放权限位 / 大页等信息）
- TLB 缓存虚拟地址，对应 PTE，目的是加速 VP 到 PP 的翻译
- Cache 缓存物理地址，对应以物理地址开头的物理块，目的是加快数据字的访问

TLB 的刷新：**每个进程都有自己的虚拟地址空间，进程间切换需要 TLB 刷新，切换到对应 VP 的解析**

- 在切换上下文后 TLB 的内容完全不对怎么办？
 - 选项 1：清空全部的 TLB
 - 选项 2：将进程 ID 与每个 TLB 相关联

要点总结

Recap

地址翻译由硬件和软件协同完成：

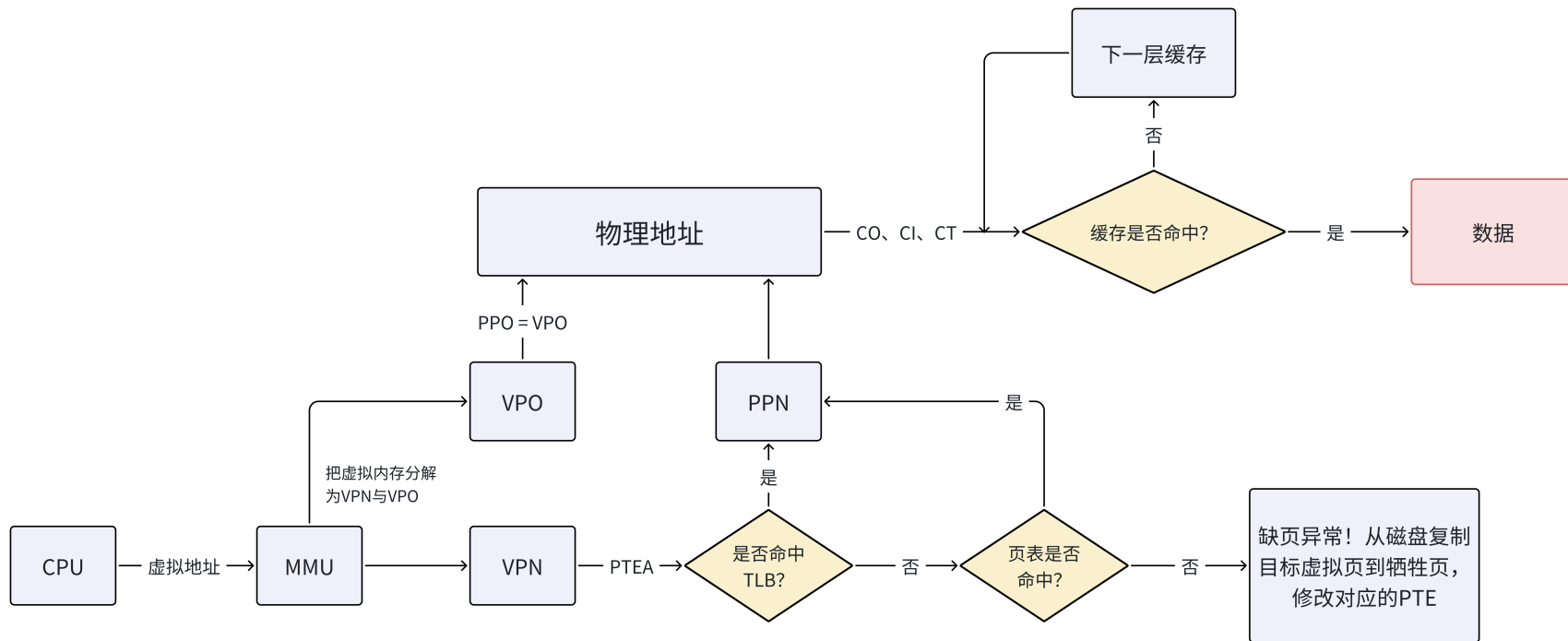
- 硬件负责：地址翻译 + 权限保护
- 软件负责：创建页表，维护页表

多级页表：

- 多级页表前 $k - 1$ 级的地址字段是 **下一级页表** 的物理页基质，第 k 级的地址字段是虚拟页对应的 **物理页** 的基址
- 当多级页表具有较好连续性时，可以使用大页模式

要点总结

Recap



酷睿 i7 的地址翻译要点

Key Points of Intel Core i7 Address Translation

页表 4KB 对齐，使得我们可以在低 12 位中设置权限位、大页位等字段，其中重要字段包括：

- **有效位** (P, Present) ，表示该页是否存在于物理内存中
- **大页位** (PS, Page Size) ，表示该页是否是大页，**只对第三层 PTE 有效**
四级页表，第三层第四层组成了大页，连续的 1024 个 4KB 的页组成了 4MB 的大页
- **引用位** (A, Accessed) ，表示该页是否被访问过，**由 MMU 在读时设置，由软件清除，为了实现替换策略**
- **脏位** (D, Dirty) ，表示该页是否被修改过，**只对最低的第四层 PTE 有效，由 MMU 在读和写时设置，由软件清除，为了确定是否要写回**

酷睿 i7 的地址翻译要点

Key Points of Intel Core i7 Address Translation

这又加强了一个要求，要求物理(页)4KB 对齐。

63	62	52	51	12	11	9	8	7	6	5	4	3	2	1	0
XD	未使用	页表物理基地址				未使用	G	0	D	A	CD	WT	U/S	R/W	P=1

OS 可用（磁盘上的页表位置）														P=0
-----------------	--	--	--	--	--	--	--	--	--	--	--	--	--	-----

字段	描述
P	子页表在物理内存中（1），不在（0）
R/W	对于子页，只读或者读写访问权限
U/S	对于子页，用户或超级用户（内核）模式访问权限
WT	子页的直写或写回缓存策略
CD	能 / 不能缓存
A	引用位（由 MMU 在读和写时设置，由软件清除）
D	修改位（由 MMU 在读和写时设置，由软件清除）
G	全局页（在任务切换时，不从 TLB 中驱逐出去）
Base addr	子页物理基地址的最高 40 位
XD	能 / 不能从这个子页中取指令

dirty bit: 是否要写回

图 9-24 第四级页表条目的格式。每个条目引用一个 4KB 子页

酷睿 i7 的地址翻译要点

Key Points of Intel Core i7 Address Translation

高速缓存 CI + CO = 12位 = VPO

这样的话：

1. 翻译地址：只使用 VPN
 2. 确立缓存：先使用 VPO (PPO) 提取出 CI 和 CO，再使用 PPN 作为 Tag 去对比
- 2 的前半段和 1 可以 **并行**，加快访存过程。

In Fact

6	6	6	5	5	5	5	5	5	5	M ¹	M-1			3	3	3	2	2	2	2	2	2	2	1	1	1	1	1	1	0	9	8	7	6	5	4	3	2	1	0
Reserved ²												Address of PML4 table (4-level paging) or PML5 table (5-level paging)												Ignored				P C W D	Ign.	CR3										
X D 3	Ignored											Rsvd.	Address of PML4 table												R	Ign.	Rsvd	A g n	P C W D	P U S T	R /	1	PML5E: present							
Ignored																														0	PML5E: not present									
X D	Ignored											Rsvd.	Address of page-directory-pointer table												R	Ign.	Rsvd	A g n	P C W D	P U S T	R /	1	PML4E: present							
Ignored																														0	PML4E: not present									
X D	Prot. Key ⁵	Ignored										Rsvd.	Address of 1GB page frame				Reserved											P A T	R	Ign.	G	1	D	A	P C W D	P U S T	R /	1	PDPT: 1GB page	
X D	Ignored											Rsvd.	Address of page directory												R	Ign.	0	A g n	P C W D	P U S T	R /	1	PDPT: page directory							
Ignored																														0	PDPT: not present									
X D	Prot. Key	Ignored										Rsvd.	Address of 2MB page frame				Reserved											P A T	R	Ign.	G	1	D	A	P C W D	P U S T	R /	1	PDE: 2MB page	
X D	Ignored											Rsvd.	Address of page table												R	Ign.	0	A g n	P C W D	P U S T	R /	1	PDE: page table							
Ignored																														0	PDE: not present									
X D	Prot. Key	Ignored										Rsvd.	Address of 4KB page frame												R	Ign.	G	P A T	D	A	P C W D	P U S T	R /	1	PTE: 4KB page					
Ignored																														0	PTE: not present									

Figure 4-11. Formats of CR3 and Paging-Structure Entries with 4-Level Paging and 5-Level Paging

内核虚拟内存

Kernel Virtual Memory

`task_struct`：是 Linux 内核中用于描述一个进程的主要数据结构。

包含或者指向内核运行该进程所需的所有信息（例如，PID、指向用户栈的指针、可执行目标文件的名字，以及程序计数器）

`mm_struct`：描述了虚拟内存的当前状态。

- `pgd`：page global directory，指向第一级页表（页全局目录）的基址，`pgd` 存放在 CR3 寄存器中
- `mmap`：memory map，指向一个 `vm_area_struct` 链表，每个 `vm_area_struct` 描述了当前虚拟地址空间中的一个区域

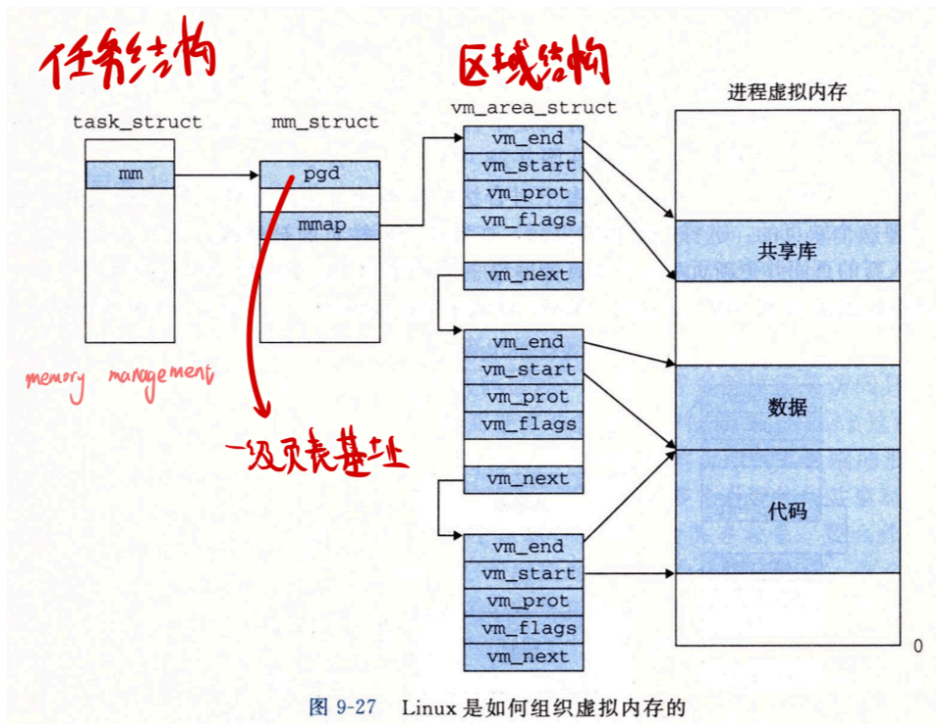


图 9-27 Linux 是如何组织虚拟内存的

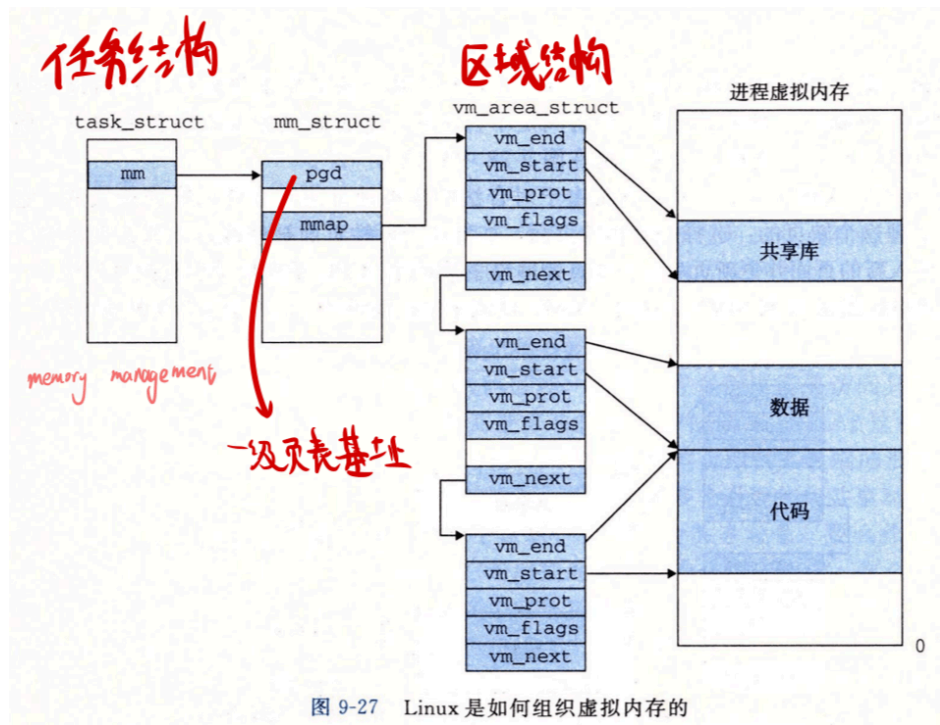
内核虚拟内存

Kernel Virtual Memory

`vm_area_struct` 描述了虚拟地址空间中的一个区域。

区域：通常指虚拟地址空间中一段连续的内存范围。

- `vm_start`：指向这个区域的起始地址
- `vm_end`：指向这个区域的结束地址
- `vm_prot`：描述这个区域内含有的所有页的读写许可权限
- `vm_flags`：描述这个区域的属性（例如是否共享等）
- `vm_next`：指向链表中的下一个区域结构



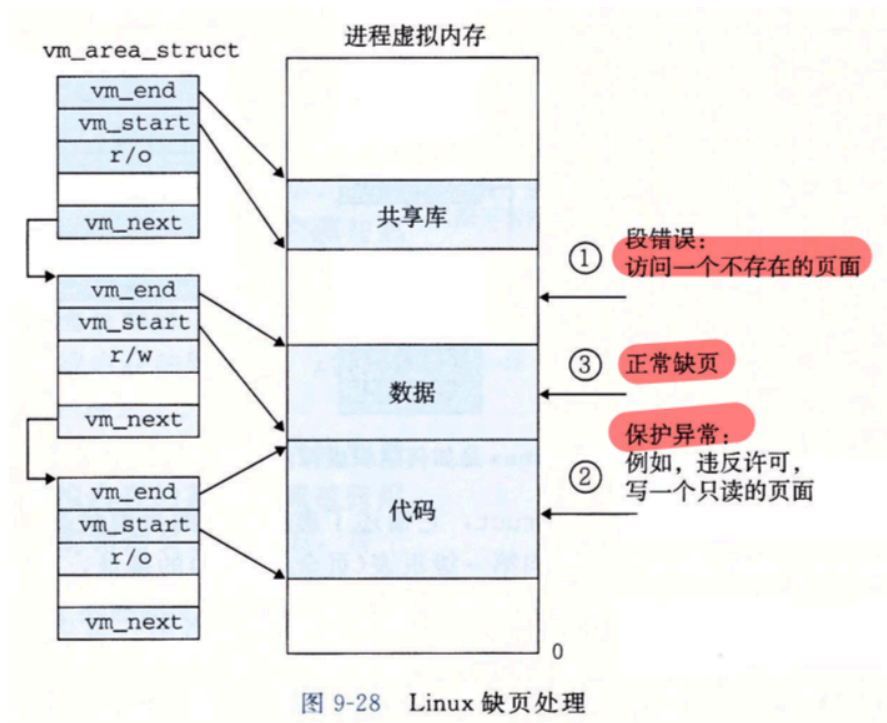
缺页异常处理程序

Page Fault Exception Handler

1. 检查虚拟地址 VA 是否合法：看它是否在某个段中。遍历比较 `vm_area_struct` 链表的子节点中从 `vm_start` 到 `vm_end` 的值，看 VA 是否在某个 `vm_start` 和 `vm_end` 之间。
 - 不合法：触发 **段错误**（页不存在于段中，就说明他不是合法的虚拟地址，不能被索引），终止进程。
2. 检查对虚拟地址 VA 的访问是否合法：确定所在段后，根据 `vm_prot` 字段检查访问权限，判断是否和操作类型匹配。
 - 不合法：触发 **段错误**，进入缺页异常处理程序，可能会修复（COW），亦可能直接终止进程
3. 如果上述检查均通过，则确定了这个缺页是由于对合法的虚拟地址进行合法的操作造成的
 - 执行正常的缺页处理
 1. 处理程序会选择一个物理牺牲页，如果牺牲页被修改了，则进行写回；
 2. 将虚拟地址 VA 对应的虚拟页写入对应的物理页中，修改对应的页表；
 3. 从处理程序返回

缺页异常处理程序

Page Fault Exception Handler



内存映射

Memory Mapping

内存映射：是把一个虚拟内存区域和 **磁盘上的一个对象**（例如文件）关联起来，以 **初始化这个虚拟内存区域** 的过程。

内存映射

延迟加载：在程序访问某个内存页时，才会从磁盘读取相应的数据到物理内存中。未访问的页不会占用物理内存。

内存共享：多个进程可以共享同一个虚拟内存区域，从而节省物理内存。

文件与内存同步：当内存中的数据被修改时，会立即同步到磁盘上的文件中，以保证数据的一致性。

减少 I/O 操作：通过内存映射，可以减少对磁盘的访问次数，提高程序的执行效率。

加载

立即加载：加载的定义就是将磁盘上的数据复制到物理内存中。

私有内存：每个进程都有自己独立的内存空间，即使访问相同的文件，内存也不会共享。

文件与内存不同步：内容加载到内存后，所有修改都是在内存中进行的，除非需要显式地写入文件，才能将修改同步到磁盘上。

恒定 I/O：每次访问文件都需要磁盘 I/O 操作，速度相对较慢。

内存映射

Memory Mapping

虚拟内存区域可以映射到两种类型的对象中的一种：

普通文件

- 一个区域可以映射到一个普通磁盘文件的连续部分
- 比如一个可执行文件，分成页大小的数据块
- 如果区域比文件区要大，那么就用零来填充这个区域的余下部分

匿名文件

- 由内核创建，全是二进制 0 构成
- 匿名文件并不真实存在，只是一个 trick，允许我们创建一个全是 0 的页
- **创建时，磁盘和内存之间没有任何实际的数据传送**
- 换出时肯定有磁盘和内存之间的数据传送

交换空间

Swap space

交换空间：是 **磁盘** 上的一个区域，用于存储那些被换出的页。

交换区可以被看作是物理内存的扩展，它提供了一个“缓存”以便在物理内存不足时存储更多的页。

虚拟内存系统允许进程使用比实际物理内存更多的内存。

1. 对于一个正在运行的进程，他修改过后的页被存放在物理内存中，但是物理内存总有放满的一天。
2. 当它被选中为牺牲页牺牲的时候，肯定不能放回原来磁盘中的文件了（因为你此时没有显式地使用 `write` 等系统调用写回文件，要保证磁盘上的原始文件没变，要不然再来一个程序加载这个文件就坏事了）。
3. 于是只好将这些被修改过后的内容放到磁盘中一个统一的地方（Page-out），这个地方称为交换区（swap space）。

交换区不一定是连续的，保存了进程被修改后的数据。可以把交换空间理解成 RAM 在磁盘上的坚实后盾。

书上提到交换区大小限制了当前运行的进程能够分配的虚拟页总数，事实上应该说 **物理内存 + 交换区** 的大小限制了当前运行的进程能够分配的虚拟页总数。

共享对象

Shared Objects

虚拟内存系统承诺要给每个进程一个抽象：他们有自己的私有虚拟内存地址，不会被其他进程所修改。

但是有时候需要共享一些段，比如说共享库（shared library）或者匿名页。

虚拟内存区域中通过 `vm_flags` 确定该虚拟内存区域中的虚拟页是否在进程之间共享。

1. 共享区域：各个进程在该区域中的操作是 **互相可见** 的，**且变化也会反映在磁盘上**
2. 私有区域：各个进程在该区域中的操作是 **互相不可见** 的，**也不会反映到磁盘上**

共享对象

Shared Objects

1. 在左边的进程中，磁盘上的一个对象通过内存映射与该进程的一个共享段关联起来，使得进程共享段的 PTE 指向磁盘上的文件，当引用该对象时，就会将对应的虚拟页加载到物理页中。
2. 右边的进程也想要把该对象与自己的一个共享段关联起来。当进程 2 对对象引用时，内核会发现左边的进程已经将对象的物理页加载进内存，因此只需要将进程 2 的 PTE 指向相同的物理页即可。

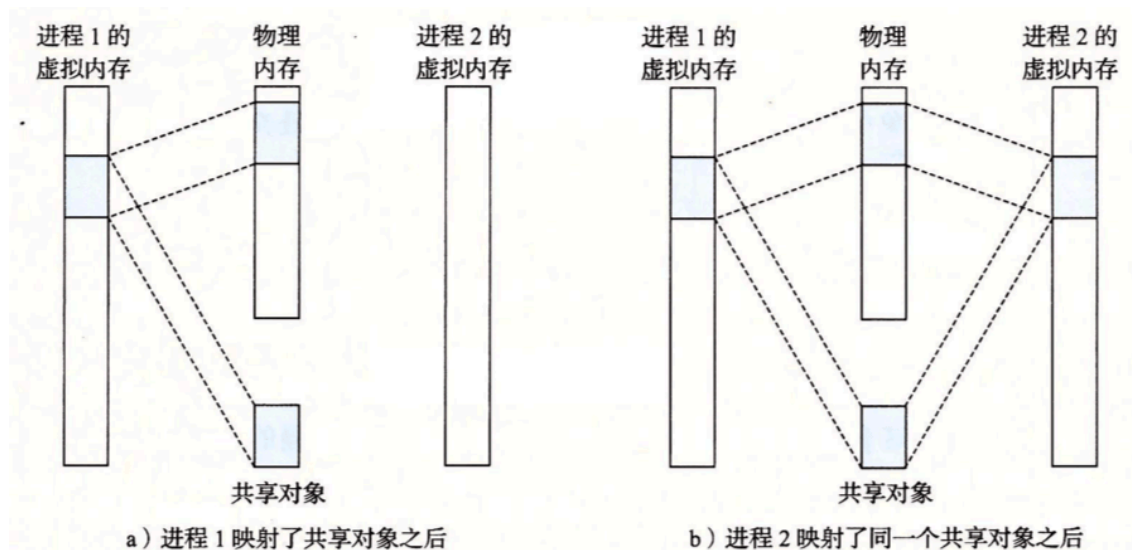


图 9-29 一个共享对象(注意，物理页面不一定是连续的)

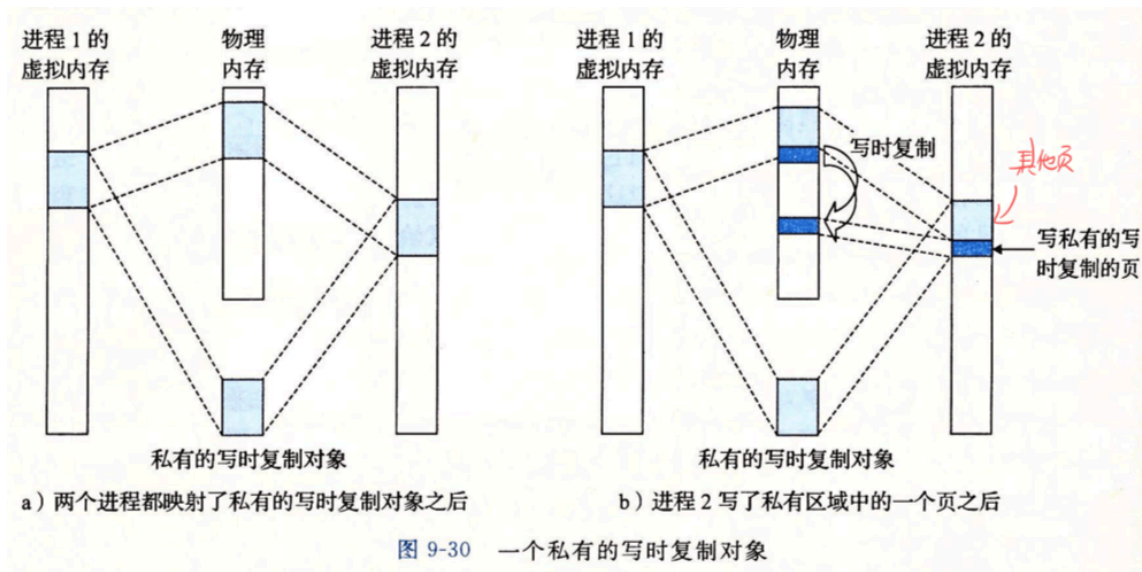
私有对象的写时复制

Copy On Write, COW

对于私有对象，使用一种叫做写时复制的巧妙技术。

1. 前面与共享对象一样，未发生修改时，物理内存只保存同一个对象副本。同时每个进程的 PTE 都是 **只读** 的
2. 当右边进程想修改时会触发 **保护故障** (protect fault)
3. 处理程序就会在物理内存中创建一个该物理页的副本
4. 然后让右边进程对该对象的 PTE 指向新的副本，并设置该虚拟页具有写权限，然后处理程序返回，从写指令重新运行。

注意这时左边进程的 PTE 还是指向最初的地方。



Fork 和 Execve

fork & execve

fork

```
pid_t fork(void);
```

1. 创建一个具有相似但独立虚拟地址空间的子进程，并分配一个唯一的 PID
2. 内核会复制父进程的 `mm_struct`、`vm_area_struct` 与页表
3. 两个进程的页都设置为 **只读** 的，**且所有区域都标记为私有的写时复制**
4. 当父子进程都没有对页进行修改时，父子进程是共享相同的物理内存的
5. 当其中一个进程对页进行修改时，就会对该页进行写回复制，并为该页赋予写权限，并更新进程对应的页表

execve

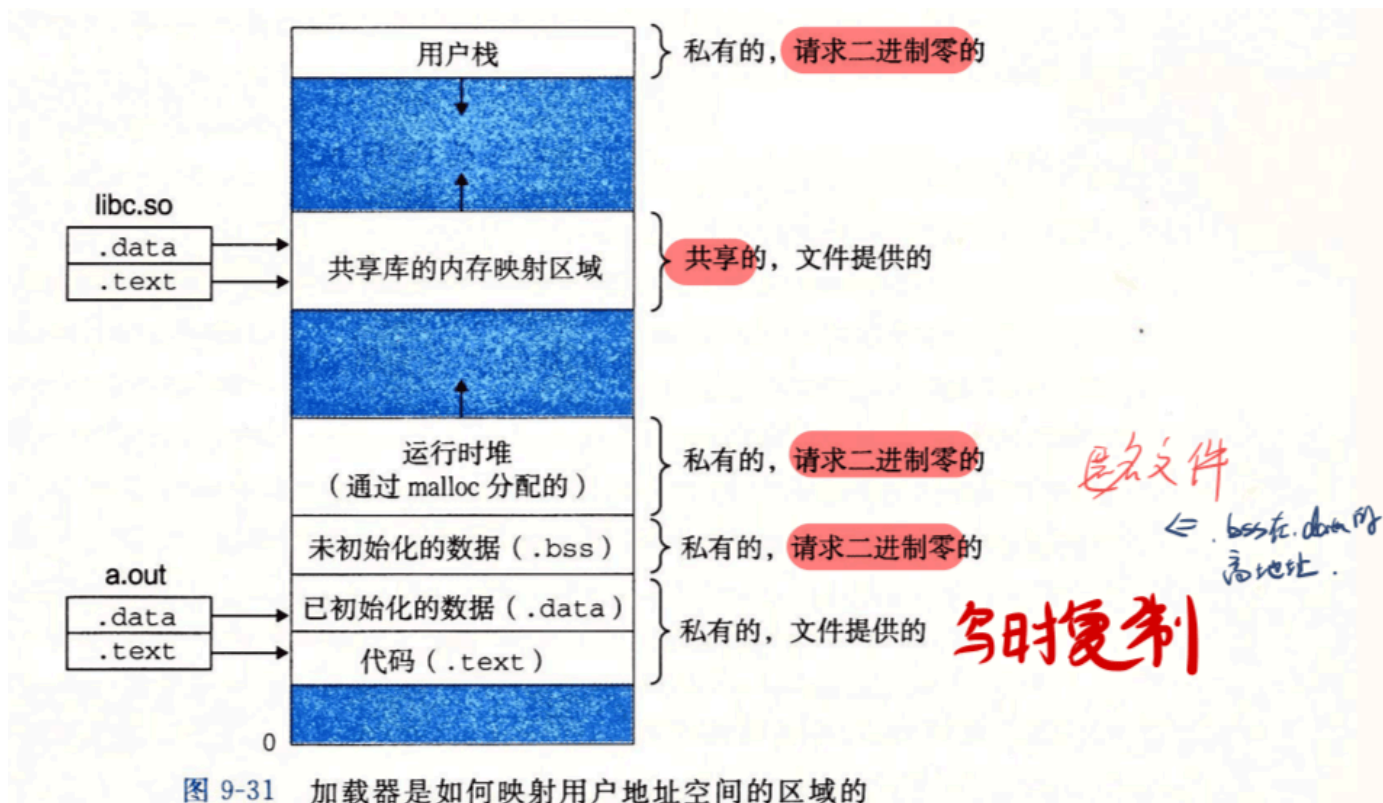
```
int execve(const char *filename, const char *argv[], cons
```

- 
1. **删除已存在的用户区域**：当前运行的程序就会变为 `filename` 了，需要删除原先的 `vm_area_struct` 和页表
 2. **映射私有区域**：为新的程序创建新的区域结构，建立内存映射，`bss`、`stack`、`heap` 都会映射到匿名文件
 3. **映射共享区域**：如共享库动态链接
 4. **设置程序计数器**：最后设置程序计数器指向代码段的入口点

类似于一个全新的加载程序的过程

加载器如何映射用户地址空间区域

How Loader Maps User Address Space Regions



Mmap

mmap Function

```
void *mmap(void *start, size_t length, int prot, int flags, int fd, off_t offset);
```

功能：使用文件描述符 `fd` 指定一个磁盘文件，该函数会将该磁盘文件中偏移 `offset` 处开始的 `length` 个字节的对象映射到虚拟内存中的 `start` 处（推荐，但不是必须）。

- `prot`：指定映射区域的访问权限，对应区域结构中的 `vm_prot`
 - `PROT_EXEC`：可执行
 - `PROT_READ`：可读
 - `PROT_WRITE`：可写
 - `PROT_NONE`：不可访问
- `flags`：指定映射区域的各种属性，对应区域结构中的 `vm_flags`

返回：若成功时则为指向映射区域的指针，若出错则为

`MAP_FAILED(-1)`

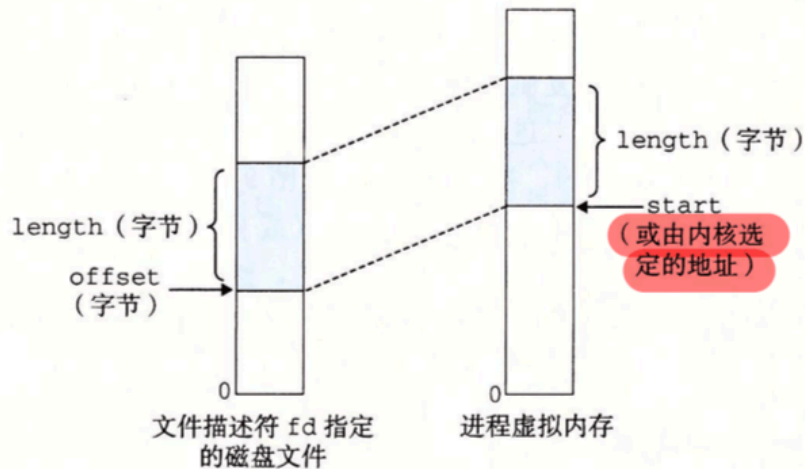


图 9-32 mmap 参数的可视化解释

动态内存分配

Dynamic Memory Allocation

下一节的内容

基本上就是堆的相关内容，大家做 mallocclab 一定会狠狠体验到的。

Emphasis

Notes

■ Note Link

- 虚拟内存
 - VM 基本概念
- VM 基本思想
 - 物理寻址
 - 虚拟寻址
 - VM 意义
- VM 作为工具
 - VM 作为缓存的工具
 - 虚拟页
 - 页表
 - 效率保证
 - VM 作为内存管理的工具
 - VM 作为内存保护的工具
- 地址翻译
 - 符号表
 - 宏观理解
 - 微观理解
 - 页表
 - 页表+Cache
 - 页表+TLB+Cache
 - 多级页表+TLB+Cache
- 翻译细节
 - Cache
 - TLB
 - 页面大小
- 例子：内存系统
 - 简单例子
- Core i7
 - 系统结构
 - 地址翻译
 - PTE 结构
 - 内存管理
- 内存映射
 - 交换空间
 - 相关函数

虚拟内存

VM 基本概念

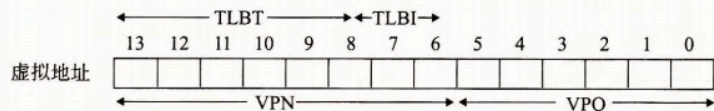
- **PA: Physical Address** 物理地址, n 位
 - PPN: Physical Page Number 物理页号, $m - p$ 位
 - PPO: Physical Page Offset 物理页面偏移, p 位
- **VA: Virtual Address** 虚拟地址, m 位
 - VPN: Virtual Page Number 虚拟页号, $n - p$ 位
 - VPO: Virtual Page Offset 虚拟页面偏移, p 位
- **MMU: Memory Management Unit** 内存管理单元
- 地址空间: 一个非负整数地址的有序集合 $\rightarrow$ 一个进程可用于寻址内存的一套地址集合
 - 线性地址空间: $\{0, 1, 2, \dots\}$
 - VAS: 虚拟地址空间 $\{0, 1, 2, \dots, N - 1\}$, $N = 2^n$
 - PAS: 物理地址空间 $\{0, 1, 2, \dots, M - 1\}$, $M = 2^m$
- **VP: Virtual Page** 虚拟页, 大小 $P = 2^p$
- **PP: Physical Page** 物理页/页帧, 大小 $P = 2^p$
- **PTE: Page Table Entry** 页表条目, 页表将虚拟页映射到物理页
 - PTBR: **Page Table Base Register** 页表基址寄存器
 - PTEA: 页表条目地址
- **TLB: Translation Lookaside Buffer** 快表, 针对PTE的缓存, 存储在MMU中
 - TLBT: TLB索引, $n - p - t$ 位

Homework Review

9.11

- 9.11 在下面的一系列问题中，你要展示 9.6.4 节中的示例内存系统如何将虚拟地址翻译成物理地址，以及如何访问缓存。对于给定的虚拟地址，请指出访问的 TLB 条目、物理地址，以及返回的缓存字节值。请指明是否 TLB 不命中，是否发生了缺页，是否发生了缓存不命中。如果有缓存不命中，对于“返回的缓存字节”用“-”来表示。如果有缺页，对于“PPN”用“-”来表示，而 C 部分和 D 部分就空着。

虚拟地址：0x027c

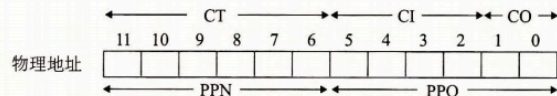


位	标记位	PPN	有效位	标记位	PPN	有效位	标记位	PPN	有效位	标记位	PPN	有效位
0	03	-	0	09	0D	1	00	-	0	07	02	1
1	03	2D	1	02	-	0	04	-	0	0A	-	0
2	02	-	0	08	-	0	06	-	0	03	-	0
3	07	-	0	03	0D	1	0A	34	1	02	-	0

a) TLB：四组，16 个条目，四路组相联

VPN	PPN	有效位	VPN	PPN	有效位
00	28	1	08	13	1
01	-	0	09	17	1
02	33	1	0A	09	1
03	02	1	0B	-	0
04	-	0	0C	-	0
05	16	1	0D	2D	1
06	-	0	0E	11	1
07	-	0	0F	0D	1

b) 页表：只展示了前 16 个 PTE



索引	标记位	有效位	块 0	块 1	块 2	块 3
0	19	1	99	11	23	11
1	15	0	-	-	-	-
2	1B	1	00	02	04	08
3	36	0	-	-	-	-
4	32	1	43	6D	8F	09
5	0D	1	36	72	F0	1D
6	31	0	-	-	-	-
7	16	1	11	C2	DF	03
8	24	1	3A	00	51	89
9	2D	0	-	-	-	-
A	2D	1	93	15	DA	3B
B	0B	0	-	-	-	-
C	12	0	-	-	-	-
D	16	1	04	96	34	15
E	13	1	83	77	1B	D3
F	14	0	-	-	-	-

c) 高速缓存：16 个组，4 字节的块，直接映射

9.11

A. 虚拟地址格式

13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	1	0	0	1	1	1	1	1	0	0

B. 地址翻译

参数	值
VPN	0x9
TLB索引	0x1
TLB标记	0x2
TLB命中? (是/否)	否
缺页? (是/否)	否
PPN	0x17

C. 物理地址格式

11	10	9	8	7	6	5	4	3	2	1	0
0	1	1	0	1	1	1	1	1	1	1	0

D. 物理地址引用

参数	值
字节偏移	0
缓存索引	0xf
缓存标记	0x17
缓存命中? (是/否)	否
返回的缓存字节	1

9.12

A. 虚拟地址格式

13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	1	1	1	0	1	0	1	0	0	1

B. 地址翻译

参数	值
VPN	0xe
TLB索引	0x2
TLB标记	0x3
TLB命中? (是/否)	否
缺页? (是/否)	否
PPN	0x11

C. 物理地址格式

11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	0	0	1	1	0	1	0	0	1

D. 物理地址引用

参数	值
字节偏移	0x1
缓存索引	0xa
缓存标记	0x11
缓存命中? (是/否)	否
返回的缓存字节	—

9.13

A. 虚拟地址格式

13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	0	0	1	0	0	0	0	0	0

B. 地址翻译

参数	值
VPN	0x1
TLB索引	0x1
TLB标记	0
TLB命中? (是/否)	否
缺页? (是/否)	是
PPN	一

C. 物理地址格式

11	10	9	8	7	6	5	4	3	2	1	0

D. 物理地址引用

参数	值
字节偏移	
缓存索引	
缓存标记	
缓存命中? (是/否)	
返回的缓存字节	

L20

编写程序，打印出该进程所能访问的所有用户态虚拟页的页号。

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <errno.h>
#define BUF_SIZE 1024

int main() {
    FILE *maps_file;
    char buf[BUF_SIZE];
    unsigned long long vma_start, vma_end;
    char perms[5];
    long page_size;

    // 1. 获取系统页大小
    page_size = sysconf(_SC_PAGESIZE);
    if (page_size == -1) {
        perror("sysconf(_SC_PAGESIZE) failed");
        exit(EXIT_FAILURE);
    }

    // 2. 打开当前进程的虚拟内存映射文件
```

L20

```

maps_file = fopen("/proc/self/maps", "r");
if (maps_file == NULL) {
    perror("fopen(/proc/self/maps) failed");
    exit(EXIT_FAILURE);
}
while (fgets(buf, BUF_SIZE, maps_file) != NULL) {
    // 解析 VMA 的起始地址、结束地址、权限（忽略后续无关字段）
    if (sscanf(buf, "%llx-%llx %4s", &vma_start,
        &vma_end, perms) != 3) {
        continue; // 解析失败则跳过该行
    }
    // 4. 筛选：仅保留有访问权限的 VMA（r/w/x 至少一个）
    if (strchr(perms, 'r') == NULL &&
        strchr(perms, 'w') == NULL &&
        strchr(perms, 'x') == NULL) {
        continue;
    }
    // 5. 筛选：仅保留用户态地址（区分 32/64 位系统）
    if (sizeof(void*) == 8) { // 64位系统
        if (vma_start >= 0x800000000000ULL) {
            continue;
        }
    } else { // 32位系统

```

```

// 6. 计算该 VMA 内的所有虚拟页号并打印
        for (unsigned long long addr = vma_start;
            addr < vma_end; addr += page_size) {
            unsigned long long page_num = addr / page_size;
            printf("%llu with %s\n", page_num, perms);
        }
    }
    // 检查读取过程是否出错
    if (ferror(maps_file)) {
        perror("fgets failed");
        fclose(maps_file);
        exit(EXIT_FAILURE);
    }
    // 清理资源
    fclose(maps_file);
    return 0;
}

```

Exercises

E1

15. 下列关于虚存和缓存的说法中，**正确**的是：
- A. TLB 是基于物理地址索引的高速缓存
 - B. 多数系统中，SRAM 高速缓存基于虚拟地址索引
 - C. 在进行**线程**切换后，TLB 条目绝大部分会失效
 - D. 多数系统中，在进行进程切换后，SRAM 高速缓存中的内容不会失效

E1

15. 下列关于虚存和缓存的说法中，**正确**的是：

- A. TLB 是基于物理地址索引的高速缓存
- B. 多数系统中，SRAM 高速缓存基于虚拟地址索引
- C. 在进行**线程**切换后，TLB 条目绝大部分会失效
- D. 多数系统中，在进行进程切换后，SRAM 高速缓存中的内容不会失效

答案：D↵

解析：属于中等题。考察虚拟内存和高速缓存的关系，并且和进程/线程有一定联系。↵

A 课本原话：A TLB is a small, virtually addressed cache where each line holds a block consisting of a single PTE. ↵

B 课本原话：Although a detailed discussion of the trade-offs is beyond our scope here, most systems opt for physical addressing.↵

C 线程共享虚拟地址空间，所以 TLB 中条目不会失效↵

D 课本原话：With physical addressing, it is straightforward for multiple processes to have blocks in the cache at the same time and to share blocks from the same virtual pages.↵

E2

16. 阅读下列代码。(已知文件“input.txt”中的内容为“12”，头文件没有列出)

```
void *Mmap(void *addr, size_t length, int prot, int flags,
           int fd, off_t offset);

int main(){
    int status;
    int fd = Open("./input.txt", O_RDWR);
    char* bufp = Mmap(NULL, 2, PROT_READ | PROT_WRITE ,
                      MAP_PRIVATE, fd, 0);

    if (Fork()>0){
        while(waitpid(-1,&status,0)>0);
        *(bufp+1) = '1';
        Write(1, bufp, 2); // 1: stdout
        bufp = Mmap(NULL, 2, PROT_READ, MAP_PRIVATE, fd, 0);
        Write(1, bufp, 2);
    }
    else{
        *bufp = '2';
        Write(1, bufp, 2);
    }
    close(fd);
    return 0;
}
```

在 shell 中运行该程序，正常运行时的终端输出应为：

- A. 221112 B. 222121 C. 222112 D. 221111

E2

答案：A←

解析：属于中等题。主要考察对私有对象 COW 机制及 `mmap` 函数的理解。←

程序打开文件描述符 `fd` 后，使用 `mmap` 函数创建了一块映射到文本 “`./input.txt`” 所在区域的私有虚拟内存区域，其地址存储在指针 `bufp` 中。父进程创建完子进程后，首先等待子进程结束。子进程在试图对 `*bufp` 写入 ‘2’ 时，触发了一次 COW。它在物理内存中创建一个 “`input.txt`” 文本所在页面的副本，并在新的页面里完成写入操作，写入完成后子进程 `bufp` 所在地址的第一个字节值为新写入的 ‘2’，第二个字节值为原始值 ‘2’。子进程结束后，其缓冲区内 “21” 的值正常输出。父进程在试图对 `*(bufp+1)` 写入 ‘1’，同样触发了一次 COW，`bufp` 所在地址的第一个字节为原始值 ‘1’，第二个字节为新写入的 ‘1’，故父进程输出 “11”。最后父进程重新创建一块映射到 “`input.txt`” 所在页面的虚拟内存并输出。由于上述写入都是新的物理页面下完成的，“`input.txt`” 所在页面没有发生修改，故最后一步父进程的输出为 “12”。综上，最终在终端的输出为 “221112”。←

E3

14. 假设在 64 位系统下页大小被设置为 16KB, 那么采用类似 Core i7 的地址翻译过程, 四级页表最多可以索引多少位地址空间?
- A. 64
 - B. 58
 - C. 54
 - D. 52

E3

14. 假设在 64 位系统下页大小被设置为 16KB，那么采用类似 Core i7 的地址翻译过程，四级页表最多可以索引多少位地址空间？
- A. 64
 - B. 58
 - C. 54
 - D. 52
14. B。根据 Core i7 的用法，每张页表恰好有一页，每个页表项 8 字节，所以每个 VPN 都是 $14 - 3 = 11$ 位，于是四级 VPN 和 VPO 一共 58 位。

E4

15. 为了支持 16G 的虚拟地址空间，采用 3 级页表，页大小为 1KB，页表项大小依旧为 4 字节。现在映射总大小为 1MB 的虚拟地址，其分布未知但分布的最小单位限定为 1 字节，请问实现上述映射的页表结构占用的内存至少至多分别为多大？
- A. 6KB, 4113KB
 - B. 6KB, 65793KB
 - C. 6KB, 1052688KB
 - D. 3KB, 4113KB

E4

15. 为了支持 16G 的虚拟地址空间，采用 3 级页表，页大小为 1KB，页表项大小依旧为 4 字节。现在映射总大小为 1MB 的虚拟地址，其分布未知但分布的最小单位限定为 1 字节，请问实现上述映射的页表结构占用的内存至少至多分别为多大？
- A. 6KB, 4113KB
 - B. 6KB, 65793KB
 - C. 6KB, 1052688KB
 - D. 3KB, 4113KB
15. B。第二句话表明题目中的内存是按字节寻址的。首先计算 VPO 为 10 位，每个 VPN 都是 8 位，恰好是 34 位虚拟地址。最好情况是 1MB 虚拟地址映射到连续的页，即占有 1024 页，所以需要 1024 个三级页表条目（占有 4 页），1 个二级页表条目（占 1 页）和 1 个一级页表条目（占 1 页），即需要 6KB。最坏情况是每字节都恰好映射到不同的页面，也就是占有 1024×1024 个不同的页面（可以实现）。这些页面可以用尽全部 256×256 张三级页表（比如每 16 页用一个三级页表），所以需要 $256 \times 256 + 256 + 1 = 65793$ 页，即 65 973 KB。

E5

16. 某 64 位系统，物理内存地址长度为 52 位，虚拟内存地址长度为 43 位，已知页的大小为 8KB，采用多级页表进行地址翻译，每级页表都占一页，则需要几级页表：
- A. 2 级 B. 3 级 C. 4 级 D. 5 级

E5

16. 某 64 位系统，物理内存地址长度为 52 位，虚拟内存地址长度为 43 位，已知页的大小为 8KB，采用多级页表进行地址翻译，每级页表都占一页，则需要几级页表：
A. 2 级 B. 3 级 C. 4 级 D. 5 级

16. B。和物理地址无关，VPO 为 13 位，VPN 均为 10 位，所以需要三级页表。

E6

17. 在页表条目中，以下哪个条目是由 MMU 在读和写时进行设置，而由软件负责清除：
- A. P 位，子页或子页表是否在物理内存中
 - B. G 位，是否为全局页（在任务切换时，不从 TLB 中驱逐出去）
 - C. CD 位，能/不能缓存子页或子页表
 - D. D 位，修改位，是否对子页进行了修改操作

E6

17. 在页表条目中，以下哪个条目是由 MMU 在读和写时进行设置，而由软件负责清除：
- A. P 位，子页或子页表是否在物理内存中
 - B. G 位，是否为全局页（在任务切换时，不从 TLB 中驱逐出去）
 - C. CD 位，能/不能缓存子页或子页表
 - D. D 位，修改位，是否对子页进行了修改操作

17. D。根据题目的描述就可以推断是 A 位和 D 位。

E7

18. 关于写时复制 (copy-on-write) 技术的说法, 不正确的是:
- A. 写时复制既可以发生在父子进程之间, 也可以发生在对等线程之间
 - B. 写时复制既需要硬件的异常机制, 也需要操作系统软件的配合
 - C. 写时复制既可以用于普通文件, 也可以用于匿名文件
 - D. 写时复制既可以用于共享区域, 也可以用于私有区域

E7

18. 关于写时复制 (copy-on-write) 技术的说法, 不正确的是:

- A. 写时复制既可以发生在父子进程之间, 也可以发生在对等线程之间
- B. 写时复制既需要硬件的异常机制, 也需要操作系统软件的配合
- C. 写时复制既可以用于普通文件, 也可以用于匿名文件
- D. 写时复制既可以用于共享区域, 也可以用于私有区域

18. A。未解之谜。如果将共享区域理解为 `MAP_SHARED` 则 D 错误; 否则共享库也是一种共享区域, 会发生 COW。如果参照 Wikipedia 上的解说, COW 是一种技术, 用户可以自行实现, 那么对等线程之间也可能发生 COW (例如 `string y=x` 之后修改 `y`, C++ 中用 COW 实现)。A 可能是更好的选择。

E8

10. 根据本课程介绍的 Intel x86-64 存储系统，填写表格中某一个进程从用户态切换至内核态时，和进程切换时对 TLB 和 cache 是否必须刷新。
- A. ①不必刷新 ②不必刷新 ③刷新 ④不必刷新
 - B. ①不必刷新 ②不必刷新 ③不必刷新 ④不必刷新
 - C. ①刷新 ②不必刷新 ③刷新 ④刷新
 - D. ①刷新 ②不必刷新 ③不必刷新 ④刷新

E8

10. 根据本课程介绍的 Intel x86-64 存储系统，填写表格中某一个进程从用户态切换至内核态时，和进程切换时对 TLB 和 cache 是否必须刷新。
- A. ①不必刷新 ②不必刷新 ③刷新 ④不必刷新
- B. ①不必刷新 ②不必刷新 ③不必刷新 ④不必刷新
- C. ①刷新 ②不必刷新 ③刷新 ④刷新
- D. ①刷新 ②不必刷新 ③不必刷新 ④刷新
10. A。高速缓存是物理寻址的，所以一定不用刷新。用户态和内核态的转换不会改变内存映射，而上下文切换会改变虚拟内存到物理地址的对应关系，所以 TLB 在上下文切换时要刷新。

E9

11. 已知某系统页面长 2KB，页表项 8 字节，采用多层分页策略映射 48 位虚拟地址空间。若限定最高层页表占 1 页，则它可以采用多少层的分页策略？
- A. 3 层
 - B. 4 层
 - C. 5 层
 - D. 6 层

E9

11. 已知某系统页面长 2KB，页表项 8 字节，采用多层分页策略映射 48 位虚拟地址空间。若限定最高层页表占 1 页，则它可以采用多少层的分页策略？

- A. 3 层
- B. 4 层
- C. 5 层
- D. 6 层

11. C。VPO 为 11 位，页表均占一页，则 VPN 都为 8 位，所以映射满 48 位地址空间至少需要 5 级页表。

E10

13. 在一个具有 TLB 和高速缓存的系统中，假设地址翻译使用四级页表来进行，且不发生缺页异常，那么在 CPU 访问某个虚拟内存地址的过程中，至少会访问（ ）次物理内存，至多会访问（ ）次物理内存。

- A. 0, 4
- B. 0, 5
- C. 1, 4
- D. 1, 5

E10

13. 在一个具有 TLB 和高速缓存的系统中，假设地址翻译使用四级页表来进行，且不发生缺页异常，那么在 CPU 访问某个虚拟内存地址的过程中，至少会访问（ ）次物理内存，至多会访问（ ）次物理内存。

- A. 0, 4
- B. 0, 5
- C. 1, 4
- D. 1, 5

13. B。最好情况是高速缓存和 TLB 都命中，不需要访问内存；最坏情况是都不命中，需要访问四级页表项以及主存中的数据，一共 5 次。

E11

第五题。请结合教材第九章“虚拟内存”的有关知识回答问题(15分)

IA32 体系采用小端法、32 位虚拟地址和两级页表。两级页表大小相同，页大小都是 $4\text{ KB} = 2^{12}\text{ Byte}$ ，结构也相同。TLB 采用直接映射，4 位组索引。TLB 和页表每一项格式如图所示：

31	12	11	9	8	7	6	5	4	3	2	1	0
Address of 4KB page frame		Ignored		G	P	D	A	P	P	U	R	P
					A			C	W	/	/	
					T			D	T	S	W	

部分位的含义如下：

0 (P)：1 表示存在，0 表示不存在

1 (R/W)：1 表示可写，0 表示只读

2 (U/S)：1 表示内核模式，0 表示用户模式

当系统运行到某一时刻，TLB 有效位为 1 的条目如下（未列出部分都是无效的）：

索引（十进制）	TLB 标记	内容
0	0x0400	0x0ec91313
3	0x02ff	0xd5d2bac01
5	0xd551	0x019fa42d
11	0x55a6	0xfdd3c66b
13	0x5515	0xb591926b

一级页表基址为 $0x00e66000$ ，物理内存中的部分内容如下（均为十六进制）：

地址	内容	地址	内容	地址	内容
00615000	21	00615001	2d	00615002	ee
00615003	c0	006154d0	ff	006154d1	a0
00e66001	a1	00e66002	a4	00e66003	67
00e66004	21	00e66005	57	00e66006	61
00e66d07	00	2167e000	42	2167e001	67
2167e002	9a	2167e003	7c	c0ee2000	6f
c0ee2001	d5	c0ee2002	7e	c0ee2003	48
c0ee24d1	83	c0ee24d2	ec	c0ee2d21	11
c0ee2d22	6b	c0ee2d23	82	c0ee2d24	8a

1. 将 cache 清空，访问一个在主机中的虚拟地址，TLB 命中，没有触发缺页异常，这一过程中，需要访问物理内存____次（3 分）。具体来说，如果该地址为 $y = 0xd5515213$ ，y 地址所具有的实质权限是____（多选题，选对才得分 2 分）。

①可写 ②只读 ③用户模式权限 ④内核模式权限

2. 不考虑第一小问，将 cache 清空，访问一个在主机中的虚拟地址，TLB 不命中，没有触发缺页异常，这一过程中，需要访问物理内存____次（3 分）。具体来说，如果该地址为 $x = 0x004004d0$ ，那么 x 对应的二级页表起始地址是____（填写 16 进制，例如 $0x0123000$ ，2 分），x 地址上单字节的內容是____（填写 16 进制，例如 $0x00$ ，1 分）。

3. 考虑下面计算矩阵和向量乘法代码：

```
1 int *mat_vec_mul(int **A, int *x, int n)
2 {
3     int i, j;
4     int *y = (int *)malloc(n * sizeof(int));
5     for (i = 0; i < n; i++)
6         for (j = 0; j < n; j++)
7             y[i] += A[i][j] * x[j];
8     return y;
9 }
```

① 在 64 位 Linux 机器中运行该代码，输入同一组合法的参数后，每次运行返回的向量的值都不一样，修复这一错误有一种简单的方法，将第____行改为____。（第二空填写 C 代码，每空 1 分，共 2 分）

可能用到的函数：

```
void *memcpy(void *dest, const void *src, size_t n);
void *memset(void *s, int c, size_t n);
void *calloc(size_t nelem, size_t elemsize);
void *realloc(void *ptr, size_t size);
```

② 在进行前一问的测试之前，还在 64 位 Linux 机器上进行过如下用户代码测试（输入 mat_vec_mul 的参数都非零）：

```
int *y = mat_vec_mul(A, x, n);
int z = y[0];
```

结果发生了段错误。通过 gdb 调试发现 mat_vec_mul 函数内并没有发生段错误，但是在初始化变量 z 时发生了段错误，后来发现是参数的输入有问题。写出出现这种错误的充分必要条件____（1 分）

4. Double fault: Intel 处理器中有一种特殊的异常，被称为 double fault。此异常发生表明调用某个故障 (fault) A 的处理程序后又触发了另一个故障 B。正常情况下，故障 B 会有相应异常处理程序来处理，因此两个故障 B 和 A 可以被顺序解决。但是如果处理器无法正常处理故障 B，或是处理了之后依然无法处理故障 A，就会产生 double fault，并终止 (abort)。假设除了缺页异常处理程序外，其他异常处理程序都不会产生新的故障。如果在某次缺页故障时产生了 double fault，其原因可能是____（不定项选择，都选对才得分，1 分）

- ① 运行缺页故障处理程序时，CPU 上的权限位是内核态，但所执行代码段 U/S=0
- ② 运行缺页故障处理程序时，CPU 接收到了键盘发送的 Ctrl + C 信号
- ③ 缺页故障处理程序没有加载到主存中

E11

答案: ←

1. 1 ②④ ←

2. 3 0x00615000 0x48 ←

3. (1) 4 `int *y = (int *)calloc(n, sizeof(int));` 或 ←

`int *y = (int *)Calloc(n, sizeof(int));` ←

其中 `calloc/Calloc` 的两个参数只要乘起来等于 `n * sizeof(int)` 即算对 ←

(2) `n < 0` (或其他等价表达) ←

4. ③ ←

←

←

解析: ←

1. 考察 TLB 的位划分与表项含义, 属于简单题。←

题目告知 TLB 命中且无缺页, 因此只需要访问物理页, 即访问一次物理内存 ←

根据 `y` 的值可以得到 `TLBT=0xd551`, `TLBI=5`, 查表知 `y` 在 TLB 条目的内容为 `0x019fa42d`, `R/W=0`, `U/S=1`, 因此这一页的实质权限为只读、内核模式。←

参见课本第 9.6 和 9.7 节 (P567-582) ←

←

2. 考察虚拟内存地址翻译, 属于简单+中等题。←

题目告知 TLB 未命中且无缺页, 因此需要访问页目录、二级页表和物理页, 共访问三次物理内存。←

计算得 `x` 的 `VPN1=1`, `VPN2=0`, `PPO=0x4d0`。页目录基地址在 `0x00e66000`, 因此相应页目录中所在条目地址为 `0x00e66000+1*4=0x00e66004`, 从表中按小端法读出 `0x00615721`, 因此二级页表存在, 首地址为 `0x00615000`, 这恰好也是相应二级页表中所在条目的地址, 从表中按小端法读出 `0xc0ee2d21`, 因此物理页存在, 首地址为 `0xc0ee2000`, 因此 `x` 的物理地址为 `0xc0ee2000+0x4d0=0xc0ee24d0`。从表中读出一个字节, 为 `0x48`。←

参见课本第 9.6 和 9.7 节 (P567-582) ←

3. 本题考查内存安全。←

(1) 考察 `malloc` 函数的性质, 属于中等题。←

`malloc` 函数并不会清零所分配内存区域, 因此同样的输入可能会有不同的输出, 但 `mat_vec_mul` 函数别的行都不可能产生这一效果, 因此只需要将第 4 行换为 ←

`int *y = (int *)Calloc(n, sizeof(int));` ←

`Calloc` 是一个带错误检查的、会将所分配区域置零的函数。←

参见课本第 9.11.2 节正文例子 (P610)。←

←

(2) 内存安全的综合考察, 属于难题。←

假设 `malloc` 函数分配成功, 那么 `y` 不是空指针。`y[0]` 不可能在外部访问出现段错误, 因此这种情况不成立, `malloc` 一定出现了问题, 导致 `y` 是空指针。但是此时 `mat_vec_mul` 并没有触发段错误, 说明函数内部一次对 `y` 的访问都没有, 这只有可能循环都没有进入, 因此必须有 `n<=0`, 根据题设, 参数是非零的, 所以 `n<0`。以上推导出了 `n<0` 是必要条件。←
下面推导 `n<0` 是充分条件, 即任何 `n<0` 都会导致这样的错误。为此, 只需要说明 `malloc` 函数一定分配失败。首先, 这是 Linux 64 位系统, 因此地址空间为 64 位, `size_t` 为 64 位, 用户虚拟地址空间为 48 位。注意 `n*sizeof(int)` 会将 `n` 的类型转为 `long`, 再转为 `unsigned long`, 而 `n` 只有 32 位, 所以符号扩展之后最高位至少有 33 个 1, 乘 `sizeof(int)`, 即 4 之后, 最高位还有至少 29 个 1, 因此 `n*sizeof(int)>263>248`, 超过了用户虚拟地址空间的大小, 因此无论如何, `malloc` 函数都会分配失败。←

参见课本第 9.8 节 (P576)、9.9.1 节 (P588)、2.2.8 节 (P58-59)。←

←

4. 本题考查对缺页故障、异常处理的综合理解, 属于难题。←

① 如果 CPU 权限位是内核态, 它自然可以运行用户级代码, 不会触发异常。←

② 如果缺页故障处理程序运行时, 收到外部中断, 这不属于故障, 因此不会触发。←

③ 如果缺页故障处理程序不在主存, 那么调用它会触发缺页故障, 于是这个故障又会导致调用缺页故障处理程序, 但它仍然不在主存, 故这一故障无法被解决, 于是触发 `double fault`。←
相关内容参见课本第 8.1 节 (P502-507)、9.7.2 节 (P581-582) 和 Intel 64 and IA-32 Architectures Software Developer's Manual - Volume 3: System Programming Guide 的第 6.15 节。←

E12

第五题（10 分）

某 32 位机器有 24 位地址空间，采用二级页表，一级页表中有 64 个 PTE，二级页表中有 1024 个 PTE，一级页表 4KB 对齐，PTE 最高一位是有效位，没有 TLB 和 Cache。惠楚思程序员在该机器上执行了如下代码。

```
1. int* a=calloc(10000, sizeof(int));  
2. int* b = a+5000;  
3. fork();  
4. *b=0x80C3F110;
```

（1） 假设编译后 b 的值保存在寄存器中，请问该机器页面大小为①__字节，VPN1 长度为②____，VPN2 长度为③____。

（2） 由于该执行结果不符合预期，惠楚思需要对程序执行过程进行还原。程序执行过程中硬件记录了部分内存访问记录，但访问记录并不完整。目前已知在父进程执行第四行的时候，进行了若干次物理内存的读写操作，其中第一次读内存操作从地址 0x67F0E8 读出 0x80AA32C4，另外有两次将 0x80C3F110 写入内存的操作，写入的地址分别是 0xC3F50D 和 0xAA3AD0，但写入顺序并不清楚。程序结束后变量 b 中保存的值是④____。在解析“*b”的过程中，一级页表的起始地址为②____；二级页表的起始地址为③____；物理页面的起始地址为④____。（地址均用 16 进制表示）

E12

第五题 (10 分)

某 32 位机器有 24 位地址空间，采用二级页表，一级页表中有 64 个 PTE，二级页表中有 1024 个 PTE，一级页表 4KB 对齐，PTE 最高一位是有效位，没有 TLB 和 Cache。惠楚思程序员在该机器上执行了如下代码。

```
1. int* a=calloc(10000, sizeof(int));
2. int* b = a+5000;
3. fork();
4. *b=0x80C3F110;
```

(1) 假设编译后 b 的值保存在寄存器中，请问该机器页面大小为①__字节，VPN1 长度为②__，VPN2 长度为③__。

(2) 由于该执行结果不符合预期，惠楚思需要对程序执行过程进行还原。程序执行过程中硬件记录了部分内存访问记录，但访问记录并不完整。目前已知在父进程执行第四行的时候，进行了若干次物理内存的读写操作，其中第一次读内存操作从地址 0x67F0E8 读出 0x80AA32C4，另外有两次将 0x80C3F110 写入内存的操作，写入的地址分别是 0xC3F50D 和 0xAA3AD0，但写入顺序并不清楚。程序结束后变量 b 中保存的值是④__。在解析“*b”的过程中，一级页表的起始地址为⑤__；二级页表的起始地址为⑥__；物理页面的起始地址为⑦__。(地址均用 16 进制表示)

解答題四 此题我们课上讲过了，原题有几个数据错误，条件也不够。摘取正确题目的分析如下：

1. 4096; 6; 10。送分题。

2. 0xEAB450D, 0x67F000, 0xAA3000, 0xC3F000。

考虑父进程执行 *b 的写时的读写问题。在解析 *b 时，第一步当然是找出一级页表项，因此从地址 0x67F0E8 读出 0x80AA32C4 就是从一级页表中，物理地址 0x67F0E8 的位置，读出一级页表项的内容 0x80AA32C4。根据题目对 PTE 结构的描述，我们知道 0xAA3 是二级页表页的页号，即 0xAA3000 是二级页表的起始物理地址。由于一级页表是 4KB 对齐的，所以一级页表项的物理地址就是 0x67F000。

为什么会写两次？显然，我们知道 COW 机制在起作用，这段内存首先要被复制。0x80C3F110 这个数字当然是精心设计的，必然暗藏玄机。首先，一次写肯定是写 *b，那么另一次写呢？注意这次写也是父进程，因此和子进程无关。由于 COW 后内存映射需要修改，结合后面也有类似于 0xC3F... 的物理地址，我们可以推定这个实际上就是 COW 后，b 对应的二级页表项。这样我们马上知道，其二级页表项的物理地址在 0xAA3AD0，而 b 指向的真实物理地址是 0xC3F50D (页号 0xC3F)。这时我们就可以计算出 b 的值 (即虚拟地址) 了。一级页表项的偏移是 $(0x67F0E8 - 0x67F000) / 4 = 0x3A$ ，二级页表项的偏移是 $(0xAA3AD0 - 0xAA3000) / 4 = 0x2B4$ ，VPO=0x50D，所以虚拟地址是它们的拼接。注意不是直接将 16 进制拼接，虚拟页号为 11 1010 10 1011 0100=0xEAB4，所以 b=0xEAB450D。此题页表项中的最低 12 位是何含义，属于未解之谜。

E1

19. 垃圾收集器的根节点不包括以下哪个节里的数据:

- A. text
- B. data
- C. bss
- D. stack

E1

19. 垃圾收集器的根节点不包括以下哪个节里的数据:

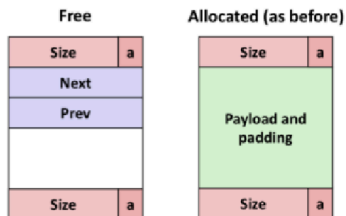
- A. text
- B. data
- C. bss
- D. stack

19. A。其他段中都可能含有指针，其引用的内存需要管理。代码段中则没有这种问题。

E2

12. 现在有一个用户程序执行了如下调用序列

```
void *p1 = malloc(16);
void *p2 = malloc(32);
void *p3 = malloc(32);
void *p4 = malloc(48);
free(p2);
void *p5 = malloc(4);
free(p3);
void *p6 = malloc(56);
void *p7 = malloc(10);
```



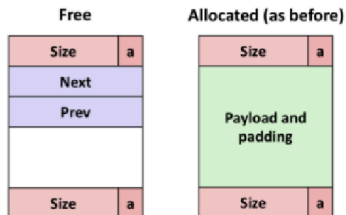
内存分配器内部使用**显式空闲链表**实现，按照地址从低到高顺序来维护空闲链表节点顺序。分配块格式参照上图。每个分配块 16 字节对齐，头部和脚部的大小都是 4 字节。分配算法采用**首次适配**算法，将适配到的空闲块的第一部分作为分配块，剩余部分变成新的空闲块，并采用**立即合并**策略。假设初始空闲块的大小是 1KB。那么以上调用序列完成后，分配器管理的这 1KB 内存区域中，**内部碎片**的总大小是____，链表里**第一个**空闲块的大小是____。

- A. 58Byte, 848Byte
- B. 26Byte, 32Byte
- C. 74Byte, 48Byte
- D. 74Byte, 16Byte

E2

12. 现在有一个用户程序执行了如下调用序列

```
void *p1 = malloc(16);
void *p2 = malloc(32);
void *p3 = malloc(32);
void *p4 = malloc(48);
free(p2);
void *p5 = malloc(4);
free(p3);
void *p6 = malloc(56);
void *p7 = malloc(10);
```



内存分配器内部使用**显式空闲链表**实现，按照地址从低到高顺序来维护空闲链表节点顺序。分配块格式参照上图。每个分配块 16 字节对齐，头部和脚部的大小都是 4 字节。分配算法采用**首次适配**算法，将适配到的空闲块的第一部分作为分配块，剩余部分变成新的空闲块，并采用**立即合并**策略。假设初始空闲块的大小是 1KB。那么以上调用序列完成后，分配器管理的这 1KB 内存区域中，**内部碎片**的总大小是____，链表里**第一个**空闲块的大小是____。

- A. 58Byte, 848Byte
- B. 26Byte, 32Byte
- C. 74Byte, 48Byte
- D. 74Byte, 16Byte

12. D。直接模拟就可以。

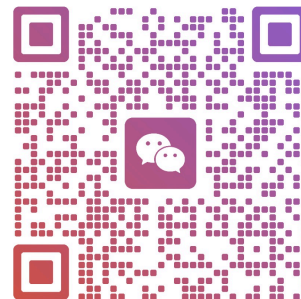
Notices

THANKS

Made by WEB-05

webrun@stu.pku.edu.cn

Reference: [WalkerCH]'s and [Arthals]'s presentations.



扫一扫上面的二维码图案，加我为朋友。